Acceso a la base de datos

Cómo se conecta la aplicación, qué permite esa cuenta y cómo conectarse uno mismo

Base: Oracle 12c Enterprise Edition 12.1.0.2.0 · 64-bit · **Servidor:** serabd (192.168.0.5:1521)
Contenedor: PDBABD · **Cuenta que usa la aplicación:** ABD
Fecha de la revisión: 4 de agosto de 2026 · **Autor:** Ing. Edgar Rojas Apaza

Lo primero, porque cambia todo lo demás.

La pregunta era «¿puedo tener acceso total a la base?». La respuesta es que **ya lo tienes**. La cuenta `ABD` —la misma que usa la aplicación y cuya contraseña está escrita en texto plano dentro del repositorio— tiene **117 privilegios activos**, entre ellos `DROP ANY TABLE`, `CREATE USER`, `ALTER DATABASE` y `GRANT ANY PRIVILEGE`.

No falta acceso. **Sobra.** Y eso es un problema de seguridad, no una comodidad. Está desarrollado en la sección 4.

1. Cómo se conecta la aplicación

La conexión no está escrita en el código: se arma en cuatro escalones, cada uno con su archivo. Así se puede cambiar de servidor sin tocar nada de programación.

```
.env las credenciales, fuera del repositorio
  ↓
src/config.js lee las variables y avisa si falta alguna
  ↓
src/db.js abre el pool de conexiones (modo Thick)
  ↓
oracledb + Instant Client → 192.168.0.5:1521/PDBABD
```

Escalón 1 — El archivo `.env`

Está en `pedidos-react/server/.env` y **no se sube al repositorio**: está en el `.gitignore`. Lo que hay dentro:

```
ORACLE_USER=abd
ORACLE_PASSWORD=••••••
ORACLE_CONNECT_STRING=192.168.0.5:1521/PDBABD
ORACLE_CLIENT_DIR=C:\instantclient_23_26
```

Para que otra persona monte el proyecto, se copia `.env.example` a `.env` y se completa. El ejemplo sí está versionado, pero sin la contraseña.

Escalón 2 — `config.js`, que valida

No se limita a leer las variables: si falta alguna obligatoria, la aplicación **no arranca** y dice cuál falta. Es preferible a arrancar y fallar más tarde con un error confuso.

```
function requerido(nombre) {
  const valor = process.env[nombre];
  if (!valor) {
```

```
    throw new Error(`Falta la variable de entorno ${nombre}. Copia .env.example a .env y completalo.`);
  }
  return valor;
}
```

Escalón 3 — `db.js` y el modo Thick

Aquí hay un detalle que costó tiempo entender. La librería `oracledb` tiene dos modos:

- **Thin** — habla con Oracle directamente. No hace falta instalar nada.
- **Thick** — se apoya en el Oracle Instant Client, un programa aparte que hay que instalar en la máquina.

Lo normal sería Thin. Pero con esta base falla:

```
NJS-116: password verifier type 0x939 is not supported in Thin mode
```

La causa: la contraseña de `ABD` está guardada con el **verificador antiguo de Oracle 10G**, que el modo Thin no acepta por ser inseguro. Por eso hay que indicar la ruta al Instant Client, lo que activa el modo Thick, que sí lo acepta.

```
if (config.oracle.clientDir) {
  oracledb.initOracleClient({ libDir: config.oracle.clientDir });
}
```

Dicho de otro modo: la aplicación necesita un rodeo porque la contraseña usa un cifrado obsoleto. Si algún día se cambia la contraseña de esa cuenta, Oracle la guardará con el verificador moderno y se podrá quitar el Instant Client y la variable `ORACLE_CLIENT_DIR`.

Escalón 4 — El pool de conexiones

No se abre una conexión por consulta, que sería lento. Se abre un **pool**: entre 1 y 10 conexiones que se prestan y se devuelven.

```
pool = await oracledb.createPool({
  user: config.oracle.user,
  password: config.oracle.password,
  connectString: config.oracle.connectString,
  poolMin: 1, poolMax: 10, poolIncrement: 1, poolTimeout: 60
});
```

Y todo el código pide la conexión con una función que **siempre la devuelve**, haya error o no. Sin esto, un error dejaría conexiones colgadas hasta agotar el pool y la aplicación se quedaría muerta:

```
async function conectar(fn) {
  let cn;
  try {
    cn = await pool.getConnection();
    return await fn(cn);
  } finally {
    if (cn) { try { await cn.close(); } catch { /* ya estaba cerrada */ } }
  }
}
```

Además, `oracledb.autoCommit = false`: nada se graba salvo que se pida expresamente. Es lo que permitió probar cosas y deshacerlas con `rollback` sin dejar rastro.

2. Qué permite esa cuenta

Esto no es una suposición: se consultó a la propia base.

2.1 Cómo se comprueba

```
-- Quién soy y dónde estoy
SELECT USER,
       SYS_CONTEXT('USERENV','DB_NAME')     AS base,
       SYS_CONTEXT('USERENV','CON_NAME')    AS contenedor,
       SYS_CONTEXT('USERENV','SERVER_HOST') AS servidor
  FROM DUAL;

-- Roles que tengo
SELECT granted_role, admin_option, default_role FROM user_role_privs;

-- TODOS los privilegios activos, incluidos los que llegan por rol
SELECT privilege FROM session_privs ORDER BY 1;

-- ¿Soy DBA?
SELECT * FROM session_roles WHERE role = 'DBA';
```

2.2 Los roles

Rol	Qué trae
`CONNECT`	Poder iniciar sesión
`RESOURCE`	Crear tablas, procedimientos y demás objetos propios
`EXP_FULL_DATABASE`	**Exportar la base entera**, incluidos esquemas ajenos
`IMP_FULL_DATABASE`	**Importar sobre la base entera**

Los cuatro con `ADMIN OPTION = YES`, o sea que además puede **concedérselos a otros**.

2.3 Los privilegios que importan

De los 117 activos, estos son los que deciden hasta dónde llega la cuenta:

Privilegio	¿Lo tiene?	Qué permite hacer
`SELECT ANY TABLE`	**SÍ**	Leer cualquier tabla de cualquier esquema
`INSERT / UPDATE ANY TABLE`	**SÍ**	Modificar datos de cualquier esquema
`DELETE ANY TABLE`	**SÍ**	Borrar filas de cualquier tabla
`DROP ANY TABLE`	**SÍ**	**Eliminar tablas completas, de quien sea**
`CREATE / ALTER ANY PROCEDURE`	**SÍ**	Cambiar la lógica de negocio de cualquier paquete
`CREATE USER`	**SÍ**	Crear cuentas nuevas
`ALTER USER`	**SÍ**	**Cambiar la contraseña de cualquiera**

Privilegio	¿Lo tiene?	Qué permite hacer
`DROP USER`	**SÍ**	Eliminar cuentas
`BECOME USER`	**SÍ**	Actuar en nombre de otro usuario
`GRANT ANY PRIVILEGE`	**SÍ**	**Darse a sí misma cualquier permiso que le falte**
`GRANT ANY ROLE`	**SÍ**	Concederse el rol DBA
`ALTER DATABASE`	**SÍ**	Cambiar la configuración de la base
`AUDIT SYSTEM`	**SÍ**	Activar y desactivar la auditoría
`SELECT ANY DICTIONARY`	**SÍ**	Ver toda la metainformación, incluidas las vistas DBA_*
`BACKUP ANY TABLE`	**SÍ**	Respaldar cualquier tabla
`SYSDBA`	**no**	Arrancar y parar la instancia. Es lo único que le falta
`Rol DBA`	**no**	No lo tiene asignado… pero podría dárselo con GRANT ANY ROLE

2.4 Comprobado en la práctica

No basta con listar privilegios; se probó leer tablas de otros esquemas sin tener permiso explícito sobre ellas:

Tabla	¿Se pudo leer?	Filas
`IBSADM.ITEMS`	**SÍ**	13.168
`SYSTEM.HELP`	**SÍ**	938
`SCOTT.EMP`	**SÍ**	14
`DBA_USERS`	**SÍ**	42 cuentas
`V$SESSION`	**SÍ**	82 sesiones activas

2.5 Qué hay dentro de la base

Con `SELECT ANY DICTIONARY` se puede inventariar todo:

Esquema	Tablas	Qué guarda
`IBSADM`	230	Administración: ítems, precios, costos
`IBSPLA`	103	Planta y producción
`IBSVENTA`	28	Ventas: pedidos, líneas, saldos
`ABD`	22	La capa propia: usuarios, personas, el paquete de operaciones
`IBSAUX`	4	Auxiliar
`SYS / SYSTEM / APEX…`	2.000+	Del propio Oracle

Cuentas abiertas en la base: `ABD, IBSADM, IBSAUX, IBSPLA, IBSVENTA, PDBADMIN, SCOTT, SYS, SYSTEM`.

Nota aparte: `SCOTT` es la cuenta de ejemplo que trae Oracle de fábrica. Que siga abierta en un servidor de producción no es buena señal; convendría cerrarla.

3. Cómo conectarse uno mismo

Cualquier herramienta que hable con Oracle sirve. Los datos son siempre los mismos:

Servidor	`192.168.0.5`
Puerto	`1521`
Servicio	`PDBABD`
Usuario	`ABD`
Contraseña	la del `.env` — no se escribe aquí a propósito

3.1 Oracle SQL Developer — lo más cómodo

Es gratuito y de la propia Oracle. Al crear la conexión:

- **Connection Type:** Basic
- **Hostname:** `192.168.0.5` · **Port:** `1521`
- Marcar **Service name** (no SID) y poner `PDBABD`

Es importante marcar «Service name». Con «SID» no conecta, porque PDBABD es una base enchufable (pluggable), no una instancia.

3.2 DBeaver — si se prefiere algo más ligero

Nueva conexión → Oracle → pestaña «Basic», mismos datos, y en «Database» elegir **Service Name**.

3.3 Por línea de comandos

```
sqlplus abd/LA_CONTRASEÑA@192.168.0.5:1521/PDBABD
```

O mejor, sin dejar la contraseña en el historial del terminal:

```
sqlplus abd@192.168.0.5:1521/PDBABD
-- la pide al entrar, y no queda escrita en ningún lado
```

3.4 Desde código, como lo hace la aplicación

```
const oracledb = require('oracledb');
oracledb.initOracleClient({ libDir: 'C:\\instantclient_23_26' });   // modo Thick

const cn = await oracledb.getConnection({
  user: 'abd',
  password: process.env.ORACLE_PASSWORD,      // nunca escrita en el código
  connectString: '192.168.0.5:1521/PDBABD'
});

const r = await cn.execute('SELECT COUNT(*) FROM IBSVENTA.PEDIDOS');
console.log(r.rows[0][0]);
await cn.close();
```

3.5 Consultas útiles para empezar a explorar

```
-- Qué esquemas hay y cuántas tablas tiene cada uno
SELECT owner, COUNT(*) tablas FROM all_tables GROUP BY owner ORDER BY 2 DESC;

-- Buscar una tabla por nombre
SELECT owner, table_name FROM all_tables WHERE table_name LIKE '%PEDIDO%';

-- Ver las columnas de una tabla
SELECT column_name, data_type, data_length, nullable
  FROM all_tab_columns
 WHERE owner = 'IBSVENTA' AND table_name = 'PEDIDOS'
 ORDER BY column_id;

-- Leer el código fuente de un paquete PL/SQL
SELECT line, text FROM all_source
 WHERE owner = 'ABD' AND name = 'PCK_OPERACIONES' AND type = 'PACKAGE BODY'
 ORDER BY line;

-- Encontrar dónde se usa una tabla en todo el código de la base
SELECT DISTINCT owner, name, type FROM all_source
 WHERE UPPER(text) LIKE '%LIN_PEDIDOS%';
```

Antes de escribir nada. Esta cuenta puede borrar tablas enteras de cualquier esquema y no hay red de seguridad. Dos costumbres que evitan disgustos:

- Escribir primero el `SELECT` con el mismo `WHERE` que va a llevar el `UPDATE` o el `DELETE`, y mirar cuántas filas salen.
- Trabajar con `autocommit` desactivado, y hacer `COMMIT` solo cuando el resultado es el esperado. Mientras tanto, `ROLLBACK` deshace todo.

4. El problema de seguridad

La contraseña de esta cuenta está en texto plano dentro del repositorio, en `src/java/hibernate.cfg.xml` y en los `constantes*.properties`.

Cuando esto se reportó en el informe de QA se clasificó como riesgo **medio**, dando por hecho que era una cuenta de aplicación con permisos acotados. **No lo es.** Con esa contraseña cualquiera que llegue al repositorio y a la red puede:

- Leer **toda** la base: ventas, costos, precios, personas
- Modificar o borrar cualquier tabla de cualquier esquema
- Cambiar la lógica de negocio dentro de los paquetes PL/SQL
- Crear cuentas, cambiar contraseñas ajenas y concederse el rol DBA
- Exportar la base entera (`EXP_FULL_DATABASE`)
- Apagar la auditoría para no dejar rastro

Se reclasifica a riesgo CRÍTICO.

4.1 Por qué está así

No es descuido de nadie en particular: es lo que pasa cuando una aplicación se conecta con la cuenta **dueña del esquema** en vez de con una cuenta propia. La cuenta dueña necesita permisos amplios para crear y mantener sus objetos, y la aplicación hereda todos.

4.2 Qué se recomienda hacer

#	Acción	Quién
1	**Cambiar la contraseña de `ABD`.** Está en un repositorio y se ha compartido; hay que darla por comprometida. De paso, la nueva se guardará con el verificador moderno y se podrá quitar el Instant Client.	Administrador de la base
2	**Crear una cuenta propia para la aplicación**, por ejemplo `APP_PEDIDOS`, con lo mínimo: `CREATE SESSION` y `EXECUTE` sobre `ABD.PCK_OPERACIONES`, más `SELECT` sobre las tablas que consulta. Nada más. Ese cambio es de una línea en el `.env`.	Administrador de la base
3	**Cuentas nominales para las personas.** Que cada quien entre con su propio usuario y no con `ABD` compartido: si algo se borra, se sabe quién fue. Hoy es imposible saberlo.	Administrador de la base
4	Sacar del repositorio las credenciales del proyecto Java y pasarlas a variables de entorno, como ya se hizo en el proyecto nuevo	Desarrollo
5	Cerrar la cuenta de ejemplo `SCOTT`, que no pinta nada en producción	Administrador de la base
6	Revisar si `ABD` necesita de verdad `GRANT ANY PRIVILEGE`, `DROP ANY TABLE` y `ALTER DATABASE`. Casi con seguridad no	Administrador de la base

4.3 Sobre pedir «acceso total»

Merece la pena decirlo con claridad: **no hace falta pedirlo, ya está concedido**. Y precisamente por eso, lo que conviene solicitar al administrador no es más acceso, sino:

- Una **cuenta nominal propia** con nombre y apellido, con permisos de lectura sobre lo que haga falta consultar.
- Que la cuenta `ABD` deje de usarse a diario y quede solo para mantenimiento del esquema.

Es mejor para todos: se sabe quién hizo qué, un error con la cuenta personal tiene menos alcance, y si alguien deja la empresa se cierra su cuenta sin tocar la aplicación.

5. Resumen

Pregunta	Respuesta
¿Cómo se conecta la aplicación?	`.env` → `config.js` → pool en `db.js` → Instant Client → PDBABD
¿Por qué hace falta el Instant Client?	La contraseña usa el verificador antiguo 10G, que el modo Thin rechaza
¿Con qué cuenta entra?	`ABD`, la dueña del esquema
¿Cuánto puede hacer esa cuenta?	117 privilegios: prácticamente todo dentro de PDBABD
¿Es DBA?	No formalmente, pero puede concederse el rol cuando quiera
¿Se puede tener acceso total?	Ya se tiene. El problema es ese, no la falta de acceso
¿Qué conviene hacer?	Cambiar la contraseña, cuenta propia para la aplicación y cuentas nominales para las personas

---

Todos los datos de este documento salen de consultar la base de producción únicamente con sentencias de lectura, el 4 de agosto de 2026. Ninguna comprobación modificó datos ni permisos. Las contraseñas no se reproducen en ningún punto del documento.